Automated Signature Identification in VLSI Metrology

Classification and Conditional Generation of Wafer-Map Failure Signatures Using Machine Learning and Deep Learning

Predictive Analysis Project

Kavinraam (23BCE1167)
Siddhish Mani (23BCE1444)
Gokul Venugopal (23BCE1003)

Vellore Institute of Technology

October 29, 2026

Abstract

This project develops and evaluates two complementary data-driven models for identifying spatial failure signatures on semiconductor wafer maps, using the public WM-811K dataset. Model 1 is a forward machine-learning (ML) classifier: 22 hand-crafted spatial features are extracted from each of the 25,519 expert-labelled wafer maps, the severe class imbalance (a 65:1 ratio between `edge-ring` and `near-full`) is addressed with SMOTETomek, and five baseline and three tuned models are compared. The tuned Random Forest achieved the best held-out performance, with accuracy 0.9592, macro F1 0.9587 and one-vs-rest macro ROC-AUC 0.9983. Model 2 is an inverse deep-learning (DL) model: a conditional variational autoencoder (cVAE) trained on the same maps resized to 32×32, which synthesizes a wafer map on demand from a failure-class label and a latent noise vector. The generator reproduces the characteristic morphology of the `center`, `donut`, `near-full`, `random` and `edge-ring` classes, supports latent-space morphing between defect types, and can export synthetic samples for data augmentation, while the thin or difluse classes (`scratch`, `loc`, `edge-loc`) remain difficult for both models. Together, the two models connect signature-to-class diagnosis with class-to-signature simulation for yield-oriented wafer inspection.

1 Introduction

Semiconductor fabrication produces wafers carrying thousands of dies, and after fabrication each die is electrically tested. The resulting wafer map records, for every die position, whether the die is absent (outside the circular wafer), good, or defective. The spatial arrangement of the defective dies is rarely random: rings, central clusters, scratches and edge arcs point to specific process excursions such as non-uniform spin coating, chuck contact, handling damage or etch non-uniformity. Recognizing these failure signatures quickly is a central task of VLSI metrology and yield engineering. Current inspection still depends heavily on manual expert review or brittle rule-based systems, and two difficulties make automation hard: rare failure patterns are heavily under-represented in the data, and the cost of missing a rare but critical pattern is far higher than that of a false alarm.

This project studies wafer-map failure signatures in both directions using the WM-811K dataset [1]. The forward direction maps a wafer map to a failure class. The inverse direction

maps a failure class (plus random noise) to a plausible wafer map. Figure 1 summarizes the two pipelines. They share the same 25,519 labelled maps, but each has its own target, model and evaluation protocol.

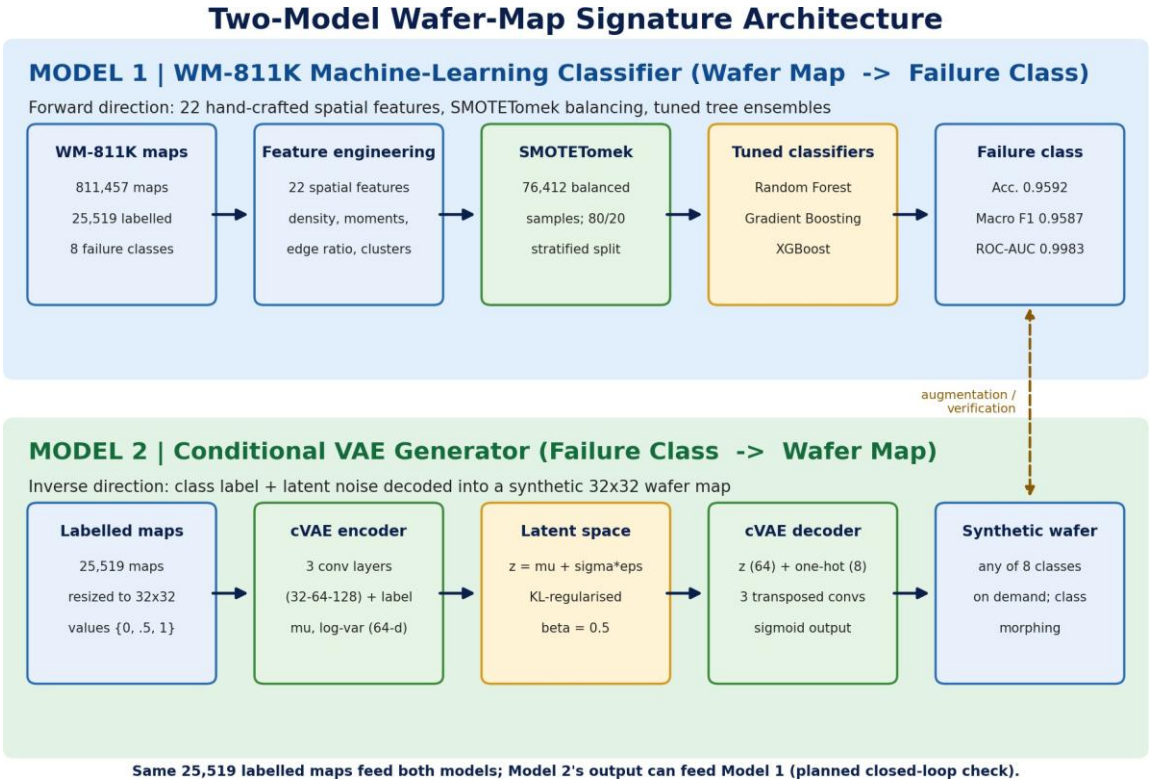


Figure 1: Architecture of the two project models. Model 1 maps 22 spatial features extracted from a wafer map to one of eight failure classes. Model 2 maps a class label and a latent vector to a synthetic 32×32 wafer map. The dashed arrow indicates the intended use of synthetic maps for augmentation and verification of Model 1.

1.1 Scope

The report covers the development and recorded results of exactly two models:

1. Model 1 — ML failure-signature classifier: feature engineering, imbalance handling (SMOTETomek) , dimensionality reduction (PCA, t-SNE) and a comparison of Logistic Regression, KNN, MLP, Random Forest, Gradient Boosting and XGBoost for eight failure classes.
2. Model 2 — DL conditional generator: a cVAE implemented in PyTorch that generates class-conditional wafer maps, interpolates between defect types and exports synthetic samples.

The scope includes dataset preparation, model configuration, evaluation procedures, saved outputs and results. A cost-sensitive attention-based CNN with explainability modules is described as the planned extension (Section 5); it is not part of the evaluated results.

1.2 Problem Statement and Objectives

Three compounding limitations motivate the work: (i) imbalanced datasets, in which rare defect classes make up well under 1-5% of samples and are ignored by standard classifiers; (ii) equal-

cost assumptions, which ignore the outsized yield impact of a missed critical defect; and (iii) an interpretability gap, since black-box outputs give engineers no spatial evidence to act on. The project has the following objectives:

1. Clean and unpack the WM-811K labels and characterize the eight failure classes through exploratory analysis.
2. Engineer interpretable spatial features and train a classifier that separates the eight signatures despite extreme class imbalance.
3. Train a conditional generative model that synthesizes realistic wafer maps for any requested failure class.
4. Provide tools for latent-space morphing between defect types and for exporting synthetic data to augment rare classes.
5. Report accuracy, macro F1, ROC-AUC, per-class behaviour and generation quality in a reproducible form.

1.3 Methodology

The methodology has four stages. First, the raw pickle of 811,457 wafer maps is loaded, nested labels such as `ttedge-ring]]` are flattened, and only the 25,519 maps carrying one of the eight failure labels are retained. Second, Model 1 converts each variable-size map into 22 spatial features, balances the classes with SMOTETomek, and trains and tunes the classifiers on an 80/20 stratified split. Third, Model 2 resizes each map to a 32×32 grid with nearest-neighbour interpolation and trains a cVAE on an 85/15 train/validation split. Finally, Model 1 is evaluated with accuracy, macro F1, per-class F1, confusion matrices and one-vs-rest ROC curves, while Model 2 is evaluated through validation loss and visual inspection of class-conditional grids, training-epoch snapshots and latent morphs.

1.4 Expected Results and Outcomes

Model 1 is expected to produce a class prediction and class probabilities for each wafer map, with the classes that have distinctive global statistics (`near-full`, `donut`, `random`) separated almost perfectly and the spatially similar localized classes (`loc`, `edge-loc`, `scratch`) being the main sources of error. Model 2 is expected to generate maps whose morphology matches the requested class, to give a continuous latent space in which one defect type can be morphed into another, and to supply synthetic samples for the rarest classes. The saved results realize most of these outcomes; where they do not, notably for `scratch` and `loc` generation, this is stated in Section 4.

1.5 Introduction Conclusion

The two-model design links diagnosis ("which failure is this map?") with simulation ("what does this failure look like?"). The remainder of the report reviews related journal research, details the dataset and both pipelines, presents the recorded outcomes, and closes with conclusions and future goals.

2 Literature Review

2.1 Wafer-Map Pattern Recognition with Classical Features

Wafer-bin-map analysis has long been used to link spatial defect clustering to process problems. Hsu and Chien [12] combined spatial clustering with data mining to extract bin-map patterns and trace them to equipment causes, and Chien et al. [13] developed a system for online detection and classification of bin-map defect patterns for manufacturing intelligence. Yu and Lu [10] used joint local and non-local linear discriminant analysis for wafer-map defect detection and recognition, and Adly et al. [11] proposed a simplified subspaced regression network for defect-pattern identification. Wu et al. [1] released the WM-811K corpus used here and showed that Radon-transform and geometry-based features support both pattern classification and similarity ranking of large wafer-map collections. Saqlain et al. [5] combined density-, geometry- and Radon-based features in a voting ensemble of classical classifiers. These studies motivate the hand-crafted spatial features and interpretable classifiers of Model 1.

2.2 Deep Learning for Wafer Maps

Convolutional networks have since become the dominant approach. Nakazawa and Kulkarni [2] trained a CNN on simulated and real wafer maps for classification and image retrieval. Kyeong and Kim [3] classified mixed-type patterns with per-pattern CNN classifiers, and Tello et al. [4] used a deep-structured model for mixed-defect recognition. Wang et al. [7] used deformable convolutions for mixed-type patterns, Kang [8] addressed rotation invariance, and Kong and Ni [9] proposed a semi-supervised and incremental framework to reduce labelling eflort. Saqlain et al. [6] designed a deep CNN specifically for an imbalanced wafer dataset. LeCun et al. [21] give the general background on deep representation learning. This literature mostly concerns the forward direction; generating wafer maps from a class label is much less common, which motivates Model 2.

2.3 Class Imbalance and Resampling

The labelled WM-811K subset is strongly long-tailed. SMOTE [14] synthesizes minority samples by interpolating between neighbours, and Tomek links [15] identify borderline pairs of opposite classes whose removal cleans the decision boundary. Batista et al. [16] evaluated combinations of over- and under-sampling and found that combining SMOTE with Tomek-link cleaning performs well on several imbalanced problems, which is the basis of the SMOTETomek step used in Model 1.

2.4 Tree Ensembles, Embeddings and Evaluation Tooling

Random forests [17] average many decorrelated decision trees and work well on tabular spatial features; gradient boosting [18] builds trees sequentially on residuals, and XGBoost [22] is an efficient implementation of it. PCA and t-SNE [19] are used to inspect the separability of the feature space, and scikit-learn [20] provides the pipelines, search procedures and metrics.

2.5 Conditional Generative Models

Variational autoencoders [23] learn a latent Gaussian representation by maximizing an evidence lower bound that balances reconstruction against a KL regularizer. Conditional variants [24] feed a class label into the encoder and decoder, so that generation can be controlled by the

label. GANs [25] are an alternative that often produces sharper images but is harder to train; the project therefore starts from the more stable cVAE and keeps a conditional GAN as a later step.

2.6 Position of the Present Project

Table 1 summarizes the journal literature and how each line of work informs this project. The project diflers from most prior work in applying both directions to the same labelled data: an interpretable forward classifier and a class-conditional generator that can supply synthetic minority samples.

Table 1: Summary of key journal studies and their relevance to this project.

Study	Contribution	Relevance here
Wu et al. 2015 [1]	WM-811K; Radon/geometry features; similarity ranking	Dataset and feature motivation
Nakazawa & Kulkarni 2018 [2]	CNN classification and retrieval	DL baseline direction
Kyeong & Kim 2018 [3]	Mixed-type patterns with CNNs	Beyond single-label cases
Saqlain et al. 2019 [5]	Voting ensemble on engineered features	Supports tree-ensemble Model 1
Saqlain et al. 2020 [6]	Deep CNN for imbalanced data	Imbalance focus
Kang 2020 [8]	Rotation-invariant CNN	Wafer maps have arbitrary orientation
Chawla et al. 2002 [14]	SMOTE oversampling	Imbalance handling
Batista et al. 2004 [16]	SMOTE + Tomek cleaning	SMOTETomek step
Breiman 2001 [17]	Random forests	Best-performing classifier
Kingma & Welling 2014 [23]	VAE	Model 2 foundation
Sohn et al. 2015 [24]	Conditional generative models	Class-conditioned decoder

3 Methods

3.1 Dataset and Provenance

Both models use the WM-811K wafer-map dataset (`LsWhD.pkl`), obtained from the Kaggle copy `qingyi/vm811k-vafer-map`. The pickle holds 811,457 wafer maps in six columns: `vaferhap`, `diesize`, `lotName`, `vaferIndex`, `trianTestLabel` and `failureType`. Each wafer map is a two-dimensional integer array of variable size in which 0 marks a position without a die, 1 a good die and 2 a defective die. The label columns are stored as nested lists (for example `ttnone]]`) and are unpacked into plain strings. Of all maps, 785,938 carry the label `none` (no failure pattern assigned) and are set aside, leaving 25,519 maps with one of eight failure labels (Table 2, Figure 2).

Table 2: Labelled WM-811K failure classes and sample counts.

Class	Characteristic morphology	Count	Share
`edge-ring`	Ring of defects along the wafer edge	9,680	37.9%
`edge-loc`	Localized defect cluster at the edge	5,189	20.3%
`center`	Compact cluster at the wafer centre	4,294	16.8%
`loc`	Localized cluster away from the edge	3,593	14.1%
`scratch`	Thin, roughly linear defect trace	1,193	4.7%
`random`	Difluse defects over the whole wafer	866	3.4%
`donut`	Ring at mid-radius with a clean centre	555	2.2%
`near-full`	Almost every die defective	149	0.6%
Total		25,519	100%

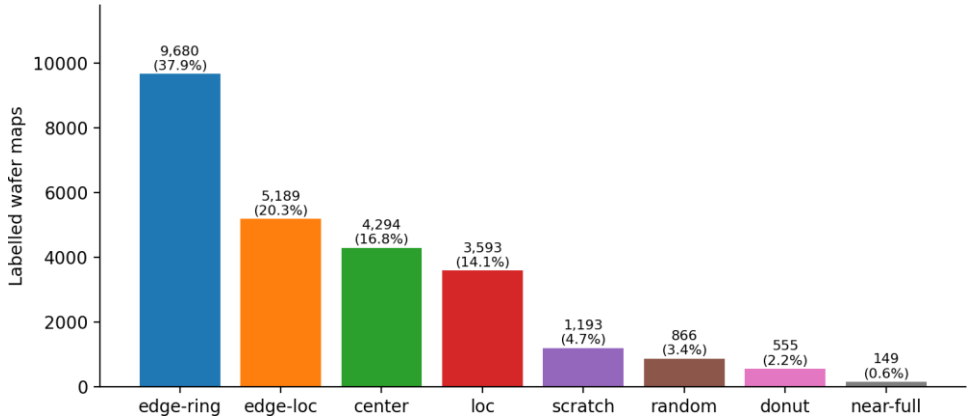


Figure 2: Class distribution of the 25,519 labelled wafer maps. The `edge-ring` to `near-full` ratio is about 65:1.

The mean wafer map of each class (Figure 3) already shows visually distinct signatures: a central hot spot for `center`, a hollow ring for `donut`, a bright border for `edge-ring`, uniformly elevated density for `near-full`, and an increased but difluse density for `random`. Representative individual maps are shown in Appendix A.

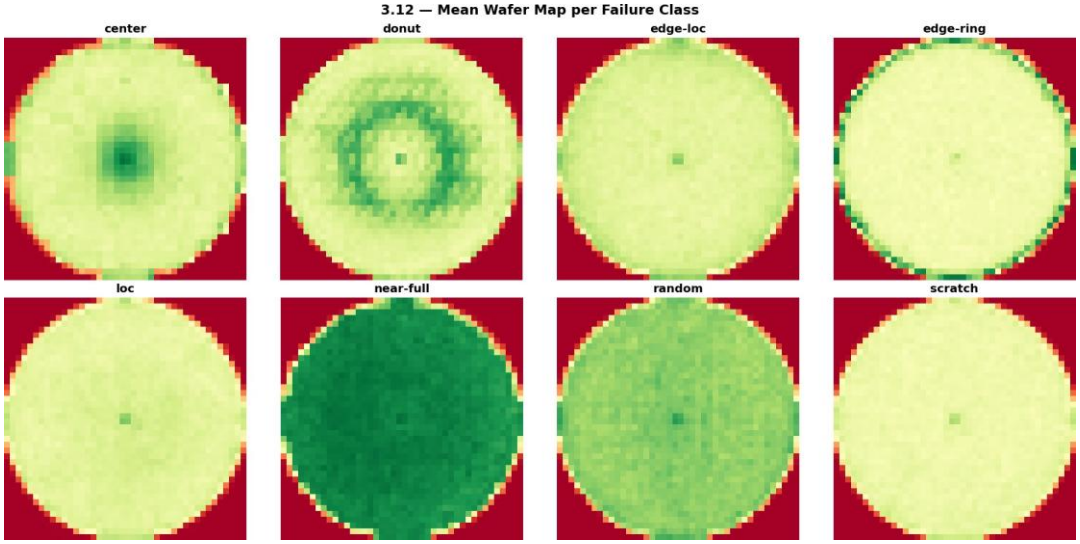


Figure 3: Mean wafer map for each failure class (darker green indicates a higher defect rate).

3.2 Model 1: Machine-Learning Signature Classifier

3.2.1 Feature engineering

Because maps difler in size, Model 1 summarizes each one with 22 interpretable features computed in parallel with Joblib. Defective-die coordinates are normalized by the map height and width before spatial moments are taken, and skewness and kurtosis are set to zero when the spread is negligible (for example a single defective die). The border zone is defined as the outer 15% of the smaller map dimension. Table 3 groups the features.

Table 3: Feature groups used by Model 1 (22 features in total).

Group	Features	Purpose
Density	`defect_density`, `yield_rate`	Overall severity
Counts	`total_dies`, `good_dies`, `defect_dies`	Absolute scale
Spatial moments	centroid, standard deviation, skewness and kurtosis of defect rows and columns	Where and how widely defects spread
done	`edge_defect_ratio`, `center_defect_count` and edge-zone defect counts	Edge versus centre patterns
Topology	`num_defect_clusters` (connected components)	Few clusters for scratch, many for random
Geometry / metadata	`map_rovs`, `map_cols`, `map_area`, `diesize`, `vaferIndex`	Lot and wafer geometry

3.2.2 Imbalance handling and dimensionality reduction

SMOTETomek oversamples every minority class with SMOTE and then removes Tomek links. After resampling, the eight classes contain between 9,410 and 9,680 samples each (76,412 in total), compared with the original range of 149 to 9,680 (Figure 4). Features are standardized with `standardscaler`. PCA with 10 components and a t-SNE embedding of a 3,000-sample

subset (perplexity 40) are used to check whether the classes separate in feature space (Figures 5 and 6).

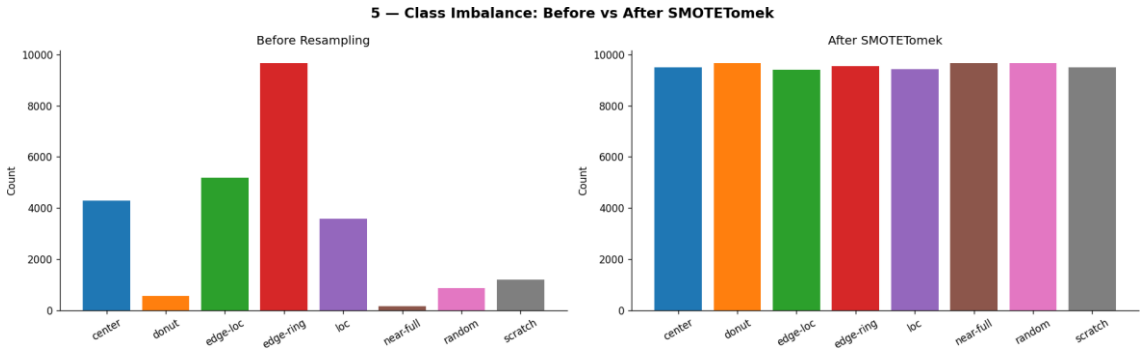


Figure 4: Class distribution before and after SMOTETomek resampling.

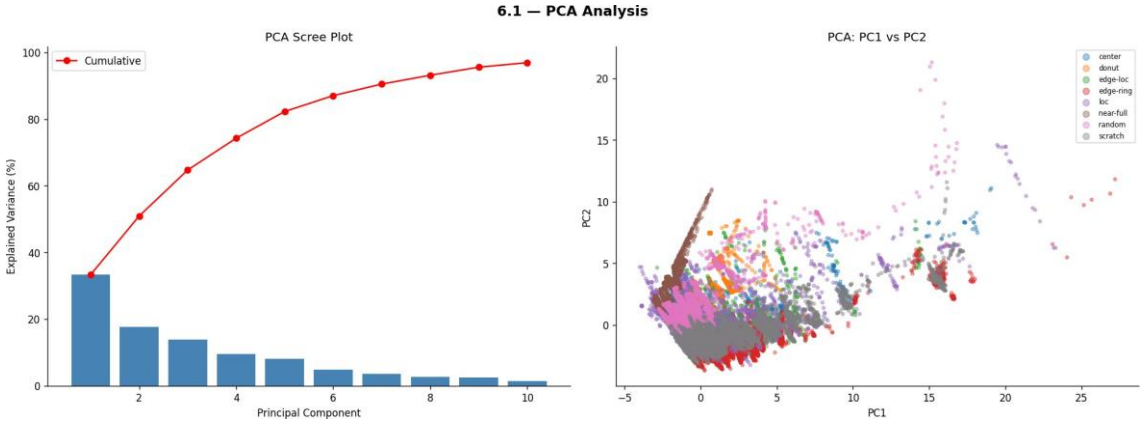


Figure 5: PCA scree plot with cumulative explained variance (left) and the first two principal components coloured by class (right).

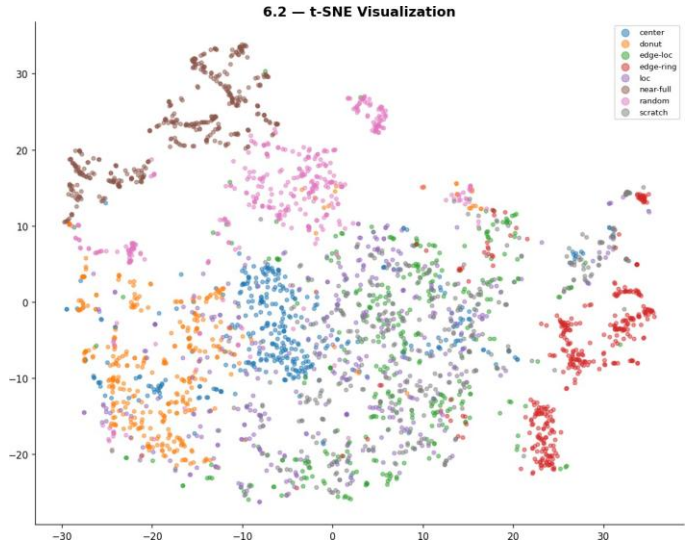


Figure 6: t-SNE embedding of the standardized feature space (3,000 sampled points).

3.2.3 Classifiers, tuning and evaluation

The resampled data are split 80/20 with stratification, giving 61,129 training and 15,283 test samples. Six baselines (Logistic Regression, KNN with $k = 7$, Random Forest with 100 trees, Gradient Boosting with 100 trees, an MLP with hidden layers of 128 and 64 units, and XGBoost) are compared with 5-fold stratified cross-validation on macro F1. The three best families (Random Forest, Gradient Boosting, XGBoost) are tuned with `RandomizedsearchCV` with 3-fold cross-validation (20, 15 and 10 candidates respectively) . The tuned Random Forest used 200 trees, maximum depth 30, `max_features=" sqrt"`, minimum split size 5, minimum leaf size 1 and no bootstrap; the tuned Gradient Boosting used 200 estimators, depth 7 and learning rate 0.2. Final evaluation on the held-out test set reports accuracy, macro and weighted F1, per-class precision/recall/F1, confusion matrices and macro one-vs-rest ROC-AUC. The random seed is fixed at 42.

3.3 Model 2: Conditional VAE for Wafer-Map Generation

3.3.1 Data preparation

Each ofthe 25,519 maps is resized to 32×32 using nearest-neighbour interpolation (`scipy. ndimage. zoom`, order 0) so that the discrete die states are preserved, and then scaled to $[0, 1]$ so that background $= 0$, good die $= 0.5$ and defective die $= 1$. The result is cached as a compressed `. npz` array of shape $(25519, 1, 32, 32)$. A random 85/15 split gives 21,691 training and 3,828 validation maps (169 and 30 batches of 128). Labels are one-hot encoded over the eight classes.

3.3.2 Architecture

The cVAE has an encoder $q_\phi(z \mid x, y)$ and a decoder $p_\theta(x \mid z, y)$. In the encoder the one-hot label is projected by a linear layer to a 32×32 map and concatenated with the wafer map as a second channel; three 4×4 stride-2 convolutions (32, 64 and 128 channels, each with batch normalization and LeakyReLU 0.2) produce a 2,048-dimensional vector from which two linear heads output the 64-dimensional mean $\boldsymbol{\mu}$ and log-variance $\log \boldsymbol{\sigma}^2$. A latent vector is sampled with the reparameterization trick

$$z = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathrm{N}(0, I). \tag{1}$$

The decoder concatenates z (64) with the one-hot label (8) into a 72-dimensional input, projects it to a $128 \times 4 \times 4$ tensor, and upsamples through three transposed convolutions ($4 \to 8 \to 16 \to 32$) with a sigmoid output.

3.3.3 Loss function and training

Training minimizes the evidence lower bound with a weighted KL term,

$$\mathrm{L} = \mathrm{BCE}(x, \hat{x}) + \beta\, D_{\mathrm{KL}}(q_\phi(z \mid x, y) \,\|\, \mathrm{N}(0, I)), \qquad \beta = 0.5. \tag{2}$$

The optimizer is AdamW (learning rate 10^{-3}, weight decay 10^{-4}) with a `ReduceLROnPlateau` scheduler (factor 0.5, patience 3), batch size 128, gradient-norm clipping at 5.0 and 25 epochs by default. The checkpoint with the lowest validation loss is kept, and a grid of generated samples for all classes is saved every five epochs.

3.3.4 Generation tools

A command-line tool (`generate.py`) loads the checkpoint and supports four actions: generating n maps for a chosen class, producing a comparison grid for all eight classes, morphing between two classes by interpolating the latent code, and exporting a labelled synthetic dataset (`.npz`) of any size per class for classifier augmentation. A sampling temperature controls diversity.

3.4 Planned Extension: Cost-Sensitive Attention Framework

The project proposal additionally specifies a four-stage inference chain for industrial deployment: a pre-trained ResNet-50 spatial feature extractor, multi-head spatial attention, a cost-sensitive classification head, and an explainability module. The cost-sensitive loss is a weighted cross-entropy in which misclassifying a rare defect as `none` carries a 100× penalty relative to a false positive, reflecting yield-loss economics. Explanations combine SHAP values, attention-map overlays and saliency maps. The proposed data pipeline has five stages: stratified sampling, weight assignment, augmentation with elastic deformation (chosen to preserve defect topology) , imbalance mitigation and normalization. This framework has not yet been trained or evaluated, and no performance numbers are claimed for it in this report.

4 Results and Discussion

4.1 Model 1: Classification Results

Table 4 lists the 5-fold cross-validated macro F1 of the baselines on the training portion, and Figure 7 shows the same comparison. Random Forest is the strongest untuned model (0.9440) , followed by XGBoost (0.9271) and the MLP (0.9132); Logistic Regression (0.7300) shows that the classes are not linearly separable in the feature space.

Table 4: Baseline 5-fold cross-validation macro F1 (training portion).

Model	CV macro F1 (mean ± std)
Logistic Regression	0.7300 ± 0.0004
KNN (k = 7)	0.8728 ± 0.0015
Gradient Boosting	0.8520 ± 0.0017
MLP (128, 64)	0.9132 ± 0.0030
XGBoost	0.9271 ± 0.0019
Random Forest	0.9440 ± 0.0027

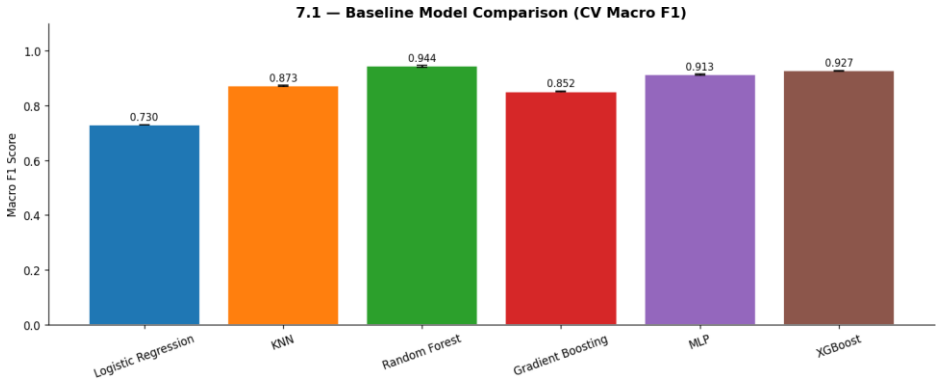


Figure 7: Baseline model comparison by 5-fold cross-validated macro F1.

After tuning, the three models were evaluated on the 15,283 held-out test samples (Table 5). The tuned Random Forest is best on every metric. Its macro one-vs-rest ROC-AUC of 0.9983 shows that the classes are well ranked and that the accuracy is not an artefact of resampling.

Table 5: Tuned-model test results (15,283 held-out samples).

Model	Accuracy	Macro F1	Weighted F1	ROC-AUC (OvR)
Tuned Random Forest	0.9592	0.9587	0.9591	0.9983
Tuned Gradient Boosting	0.9452	0.9445	0.9450	0.9963
Tuned XGBoost	0.9460	0.9452	0.9457	0.9969

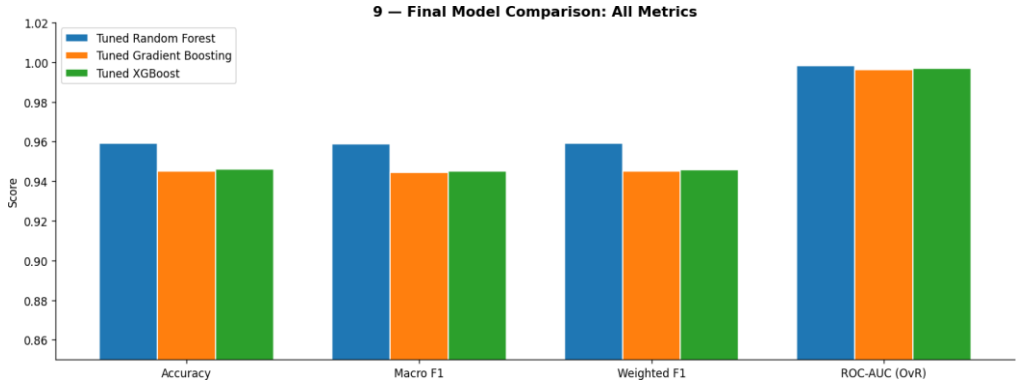


Figure 8: Final comparison of the tuned models on accuracy, macro F1, weighted F1 and ROC-AUC.

Per-class F1 (Table 6, Figure 9) shows where the difficulty lies. `near-full` is classified perfectly (F1 1.00) because its defect density is unique, and `donut`, `random` and `edge-ring` exceed 0.98. The weakest classes are `loc` (0.85-0.89) and `edge-loc` (0.88-0.91): both are localized clusters that difler mainly in their distance from the border. `scratch` has high recall but lower precision (0.91 for the Random Forest) , indicating that some localized maps are assigned to it. The most important features of the tuned Random Forest were `yield_rate`, `defect_density`, `edge_defect_ratio`, the row standard deviation of defects and `center_defect_count` (Figure 11), consistent with the physical meaning of the classes.

Table 6: Per-class F1 on the test set.

Class	Random Forest	Gradient Boosting	XGBoost
center	0.96	0.95	0.95
donut	0.99	0.99	0.99
edge-loc	0.91	0.88	0.89
edge-ring	0.98	0.97	0.98
loc	0.89	0.86	0.85
near-full	1.00	1.00	1.00
random	0.99	0.99	0.99
scratch	0.95	0.92	0.91

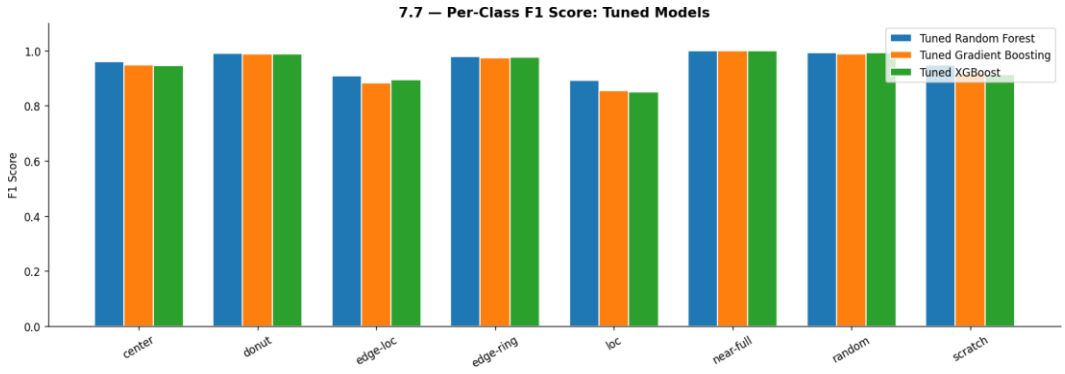


Figure 9: Per-class F1 of the three tuned models.

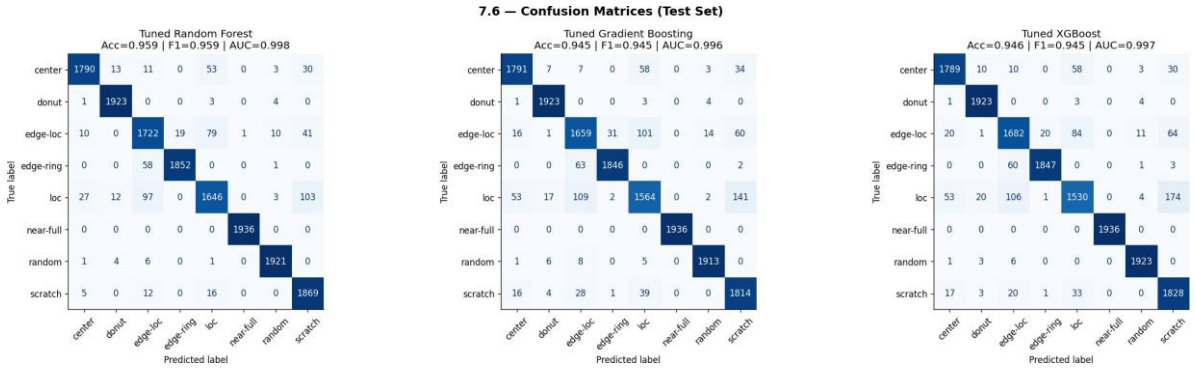


Figure 10: Test-set confusion matrices of the tuned Random Forest, Gradient Boosting and XGBoost models.

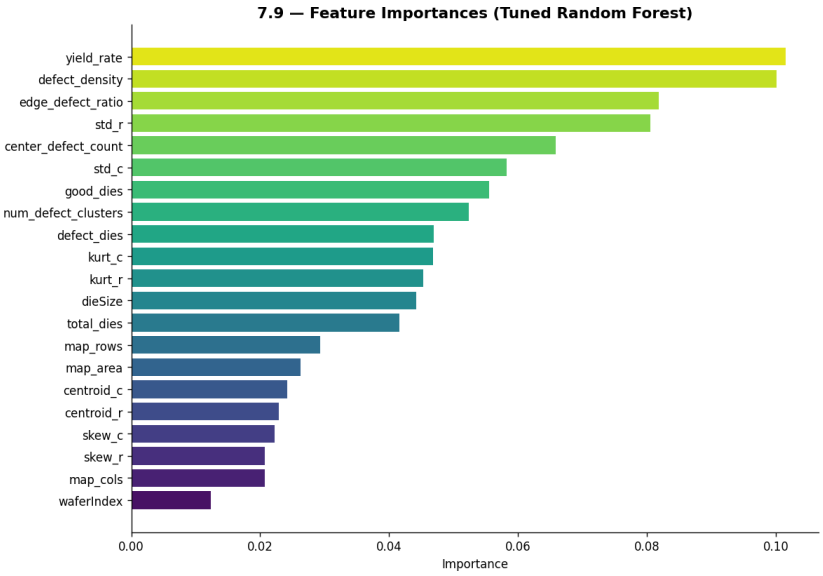


Figure 11: Feature importances of the tuned Random Forest.

A sweep of the Random Forest size (Figure 12) shows that the 3-fold macro F1 rises from 0.9189 with 10 trees to 0.9343 with 30 and 0.9377 with 100, then plateaus at 0.9382 from 200 trees onward, which supports the tuned value of 200.

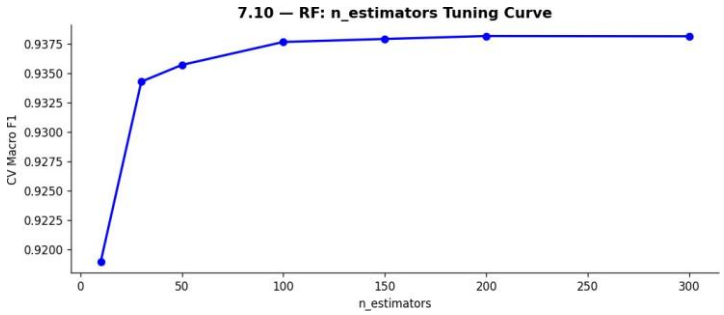


Figure 12: Random Forest `n_estimators` sensitivity (3-fold CV macro F1).

4.2 Model 2: Generation Results

The saved cVAE checkpoint corresponds to epoch 23 with a validation loss of 0.5213. Figure 13 shows the `donut` row of the training-time generation grids at epochs 2, 5, 10, 15, 20 and 25. At epoch 2 the generator produces only a faint noisy disc; by epoch 5 a ring is visible, and from epoch 10 onward the ring is stable with a clear centre, with later epochs sharpening its edges.

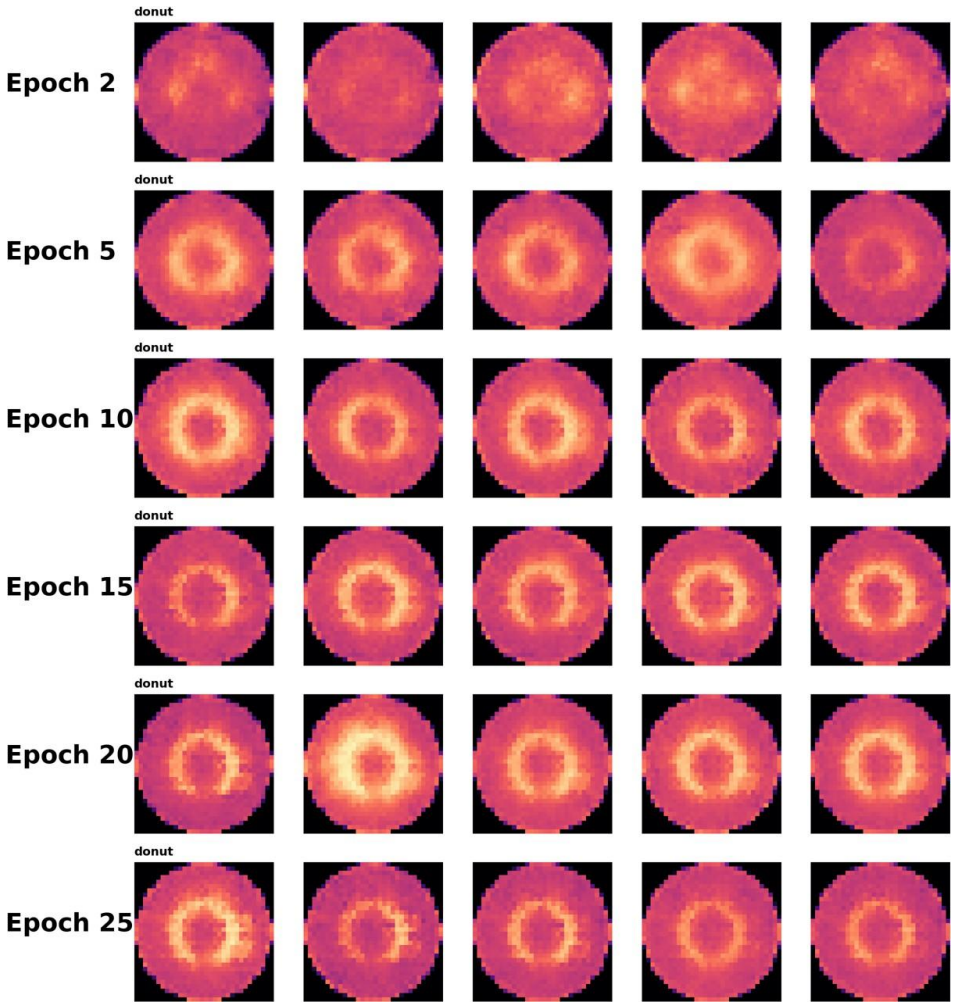


Figure 13: Five generated `donut` maps at successive training epochs.

Figure 14 shows the full class-conditional grid at epoch 25. The generated maps reproduce the signature of five of the eight classes: a single compact hot spot for `center` with varied size and intensity; a hollow ring of varied radius and thickness for `donut`; a bright ring along the border for `edge-ring`; a uniformly saturated disc for `near-full`; and a difluse raised density for `random`. The `edge-loc`, `loc` and `scratch` rows, in contrast, are close to a uniform disc with only faint structure. This matches the classifier results, in which these three classes were also the hardest, and is expected for a VAE trained with a pixel-wise BCE loss: when the position or orientation of a feature varies from sample to sample (a scratch can lie at any angle, a localized cluster at any place) , the average over many such samples is a nearly flat map. Resizing to 32×32 also removes thin one-die-wide scratches.

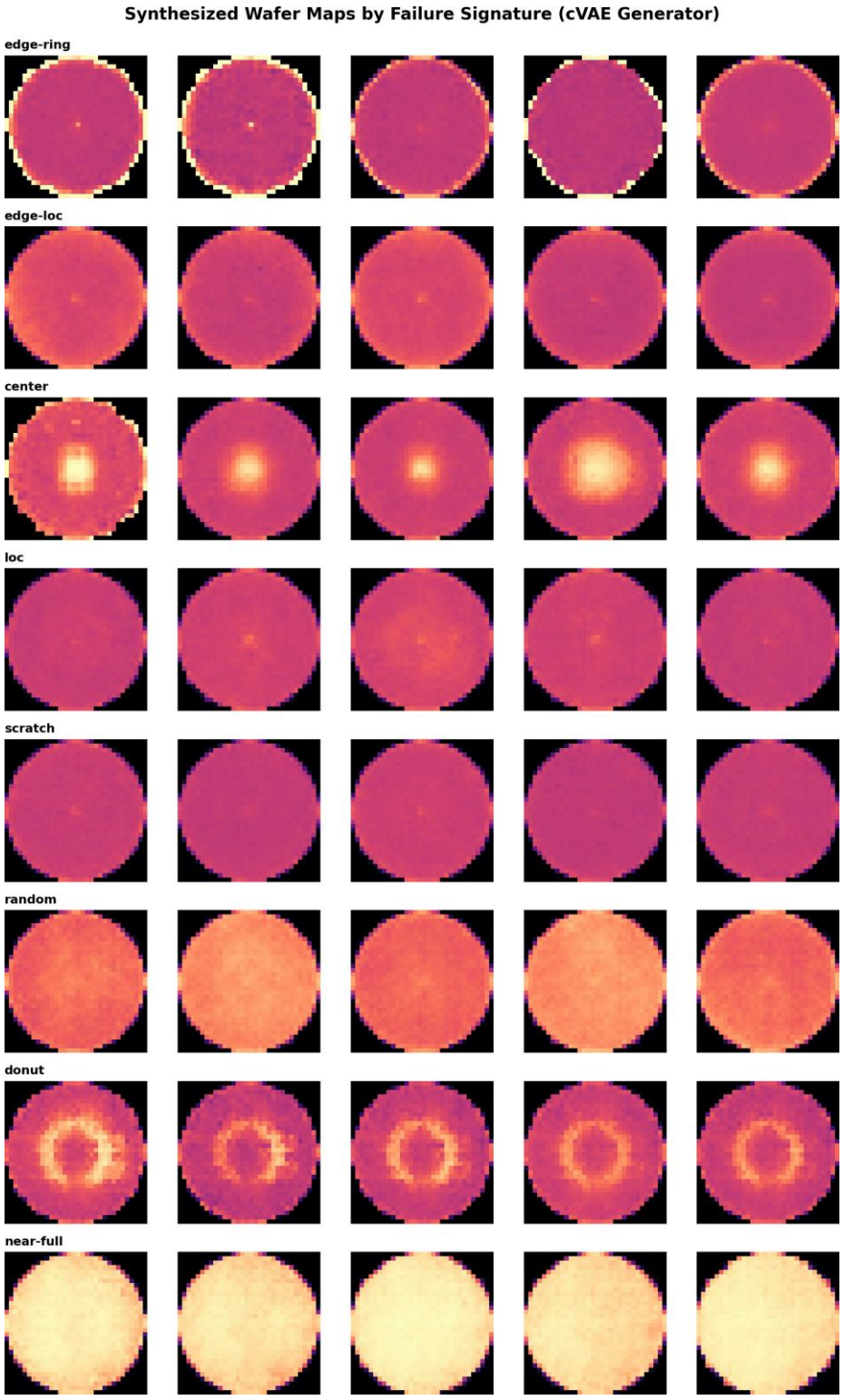


Figure 14: Synthetic wafer maps for all eight failure classes (five samples each) after 25 epochs of cVAE training.

Latent-space morphing (Figure 15) interpolates smoothly between the latent codes of two classes. The transition from `center` to `donut` changes gradually and without abrupt artefacts, which shows that the latent space is continuous and could be used to simulate gradual process drift. The endpoints of this particular strip are weaker than the epoch-25 samples in Figure 14, so it should be regenerated from the final checkpoint before being used as evidence of morphology

quality.

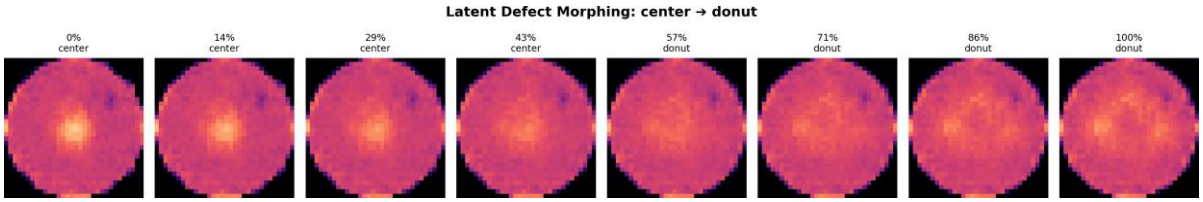


Figure 15: Latent defect morphing from `center` to `donut` in eight steps.

4.3 Comparison of the Two Models

Table 7 contrasts the two models, and Figures 8 and 9 compare the classifiers numerically. The ML model is accurate and interpretable but only labels existing maps; the DL model creates new maps, but its quality is currently assessed visually rather than with a numerical metric.

Table 7: Comparison of the ML classifier and the DL generator.

Aspect	Model 1 (ML)	Model 2 (DL cVAE)
Direction	Wafer map → failure class	Failure class + noise → wafer map
Input	22 hand-crafted features	One-hot label and 64-d latent vector
Output	Class and probabilities	32×32 synthetic map
Model	Tuned Random Forest (best)	Conditional VAE (PyTorch)
Main result	Acc. 0.9592, macro F1 0.9587, AUC 0.9983	Val. loss 0.5213; 5 of 8 classes visually faithful
Strengths	Interpretable; near-perfect on distinct classes	Class-controlled generation; morphing; augmentation export
Limitations	`loc`, `edge-loc`, `scratch` confusion; resampled test set	Blurry for `scratch`, `loc`, `edge-loc`; no FID or closed-loop score yet
Use case	Diagnosis and quality control	Augmentation, simulation, engineer training

4.4 Discussion and Limitations

Four limitations should be kept in mind. First, the classifier is evaluated on a test set drawn from the SMOTETomek-resampled data, so the split contains synthetic neighbours of training points; performance on untouched real maps, especially for the rare classes, is likely to be somewhat lower and should be confirmed by resampling only the training portion. Second, only the 25,519 labelled maps are used; the 785,938 `none` maps are not used. Third, the generator is judged visually: no FID score or closed-loop test (feeding synthetic maps to Model 1 and checking that they receive their intended label) has yet been recorded, although the codebase is designed to support it. Fourth, the generated maps are continuous density values on a 32×32 grid rather than discrete die states on the original resolution.

5 Conclusion and Future Goals

This project built two complementary models on the WM-811K wafer-map data. Model 1 converts each map into 22 spatial features, balances the eight classes with SMOTETomek, and reaches an accuracy of 0.9592, macro F1 of 0.9587 and macro ROC-AUC of 0.9983 with a tuned Random Forest; near-full, donut, random and edge-ring signatures are identified almost perfectly,

while `loc`, `edge-loc` and `scratch` remain the most confusable. Model 2 trains a conditional VAE that generates class-specific wafer maps, reproduces the morphology of the center, donut, edge-ring, near-full and random signatures, supports latent morphing, and exports synthetic datasets for augmentation. Together they cover diagnosis and simulation for yield-oriented wafer inspection.

Future goals.

1. Cost-sensitive attention network: train the planned ResNet-50 plus multi-head attention classifier with a misclassification-cost matrix, and compare it with the Random Forest on rare-class recall and yield-weighted error.
2. Explainability: add SHAP, attention overlays and saliency maps so that process engineers can validate each prediction.
3. Stricter evaluation: resample only the training portion, keep a real-data test set, and report per-class precision and recall on it.
4. Closed-loop generator validation: score synthetic maps with Model 1 and add FID; use synthetic `near-full`, `donut` and `scratch` maps to retrain the classifier and compare against SMOTETomek.
5. Better generators: train at 64×64, add a conditional DCGAN or diflusion model, and use a loss or architecture that preserves thin linear structures such as scratches.
6. Use the unlabelled data: exploit the `none` maps for semi-supervised or anomaly-detection training, and add mixed-type defect patterns.

References

[1] M.-J. Wu, J.-S. R. Jang, and J.-L. Chen, "Wafer map failure pattern recognition and similarity ranking for large-scale data sets," /£££ Trans. Semiconductor Manufacturing, vol. 28, no. 1, pp. 1-12, 2015. doi:10.1109/TSM.2014.2364237.

[2] T. Nakazawa and D. V. Kulkarni, "Wafer map defect pattern classification and image retrieval using convolutional neural network," /£££ Trans. Semiconductor Manufacturing, vol. 31, no. 2, pp. 309-314, 2018. doi:10.1109/TSM.2018.2795466.

[3] K. Kyeong and H. Kim, "Classification of mixed-type defect patterns in wafer bin maps using convolutional neural networks," /£££ Trans. Semiconductor Manufacturing, vol. 31, no. 3, pp. 395-402, 2018. doi:10.1109/TSM.2018.2841416.

[4] G. Tello, O. Y. Al-Jarrah, P. D. Yoo, Y. Al-Hammadi, S. Muhaidat, and U. Lee, "Deep-structured machine learning model for the recognition of mixed-defect patterns in semiconductor fabrication processes," /£££ Trans. Semiconductor Manufacturing, vol. 31, no. 2, pp. 315-322, 2018. doi:10.1109/TSM.2018.2825482.

[5] M. Saqlain, B. Jargalsaikhan, and J. Y. Lee, "A voting ensemble classifier for wafer map defect patterns identification in semiconductor manufacturing," /£££ Trans. Semiconductor Manufacturing, vol. 32, no. 2, pp. 171-182, 2019. doi:10.1109/TSM.2019.2904306.

[6] M. Saqlain, Q. Abbas, and J. Y. Lee, "A deep convolutional neural network for wafer defect identification on an imbalanced dataset in semiconductor manufacturing processes," /£££ Trans. Semiconductor Manufacturing, vol. 33, no. 3, pp. 436-444, 2020. doi:10.1109/TSM.2020.2994357.

[7] J. Wang, C. Xu, Z. Yang, J. Zhang, and X. Li, "Deformable convolutional networks for efficient mixed-type wafer defect pattern recognition," /£££ Trans. Semiconductor Manufacturing, vol. 33, no. 4, pp. 587-596, 2020. doi:10.1109/TSM.2020.3020985.

[8] S. Kang, "Rotation-invariant wafer map pattern classification with convolutional neural networks," /£££ Access, vol. 8, pp. 170650-170658, 2020. doi:10.1109/ACCESS.2020.3024201.

[9] Y. Kong and D. Ni, "A semi-supervised and incremental modeling framework for wafer map classification," /£££ Trans. Semiconductor Manufacturing, vol. 33, no. 1, pp. 62-71, 2020.

[10] J. Yu and X. Lu, "Wafer map defect detection and recognition using joint local and nonlocal linear discriminant analysis," /£££ Trans. Semiconductor Manufacturing, vol. 29, no. 1, pp. 33-43, 2016. doi:10.1109/TSM.2015.2497264.

[11] F. Adly, P. D. Yoo, S. Muhaidat, Y. Al-Hammadi, U. Lee, and M. Ismail, "Simplified subspaced regression network for identification of defect patterns in semiconductor wafer maps," /£££ Trans. /ndustrial /nformatics, vol. 11, no. 6, pp. 1267-1276, 2015.

[12] S.-C. Hsu and C.-F. Chien, "Hybrid data mining approach for pattern extraction from wafer bin map to improve yield in semiconductor manufacturing," /nt. J. Production £conomics, vol. 107, no. 1, pp. 88-103, 2007.

[13] C.-F. Chien, S.-C. Hsu, and Y.-J. Chen, "A system for online detection and classification of wafer bin map defect patterns for manufacturing intelligence," /nt. J. Production Research, vol. 51, no. 8, pp. 2324-2338, 2013.

[14] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," J. Artificial /ntelligence Research, vol. 16, pp. 321-357, 2002.

[15] I. Tomek, "Two modifications of CNN," /£££ Trans. Systems, Man, and Cybernetics, vol. SMC-6, no. 11, pp. 769-772, 1976.

[16] G. E. A. P. A. Batista, R. C. Prati, and M. C. Monard, "A study of the behavior of several methods for balancing machine learning training data," ACM S/GKDD £zplorations Newsletter, vol. 6, no. 1, pp. 20-29, 2004.

[17] L. Breiman, "Random forests," Machine £earning, vol. 45, no. 1, pp. 5-32, 2001.

[18] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," Annals of Statistics, vol. 29, no. 5, pp. 1189-1232, 2001.

[19] L. van der Maaten and G. Hinton, "Visualizing data using t-SNE," J. Machine £earning Research, vol. 9, pp. 2579-2605, 2008.

[20] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Machine £earning Research, vol. 12, pp. 2825-2830, 2011.

[21] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," Nature, vol. 521, pp. 436-444, 2015.

[22] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. 22nd ACMS/GKDD /nt. Conf. Knowledge Discovery and Data Mining, 2016, pp. 785-794.

[23] D. P. Kingma and M. Welling, "Auto-encoding variational Bayes," in Proc. /nt. Conf. £earning Representations (/C£R), 2014.

[24] K. Sohn, H. Lee, and X. Yan, "Learning structured output representation using deep conditional generative models," in Advances in Neural /nformation Processing Systems (Neur/PS), vol. 28, 2015.

[25] I. Goodfellow et al., "Generative adversarial nets," in Advances in Neural /nformation Processing Systems (Neur/PS), vol. 27, 2014.

A Supplementary Figures and Screenshots

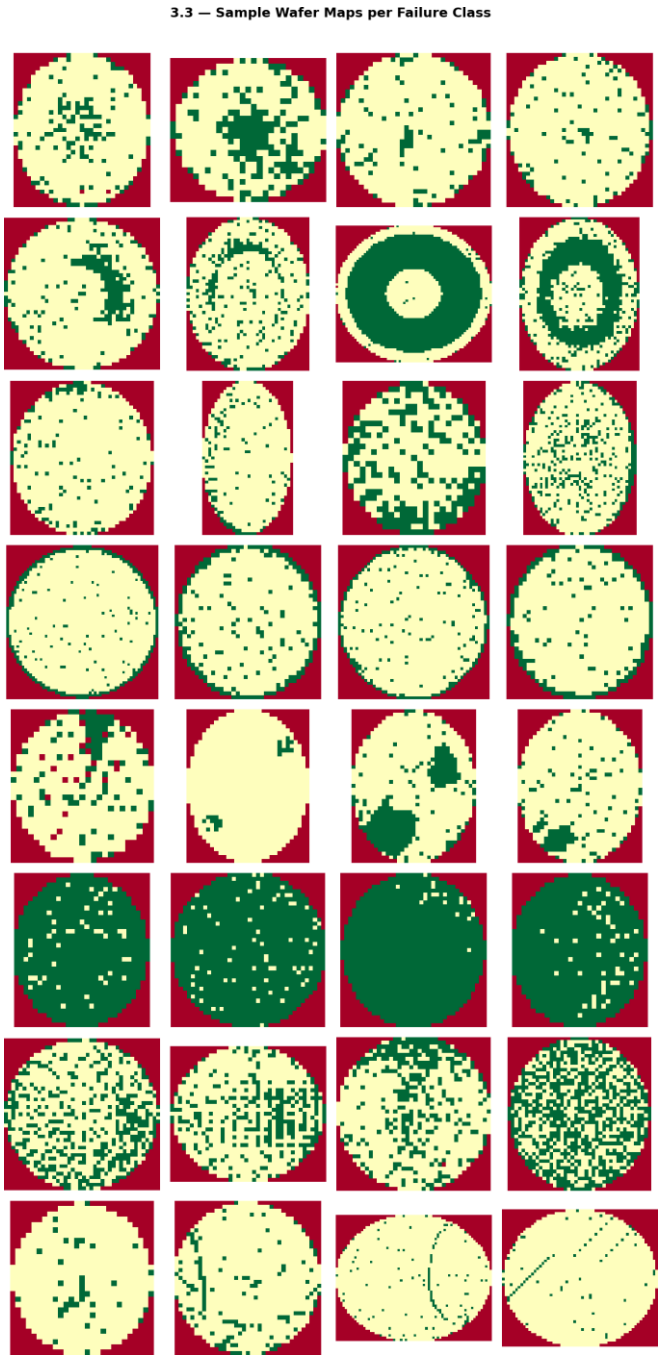


Figure 16: Representative real wafer maps for the eight failure classes (green: good die, red: defective die, yellow: background as rendered in the notebook).

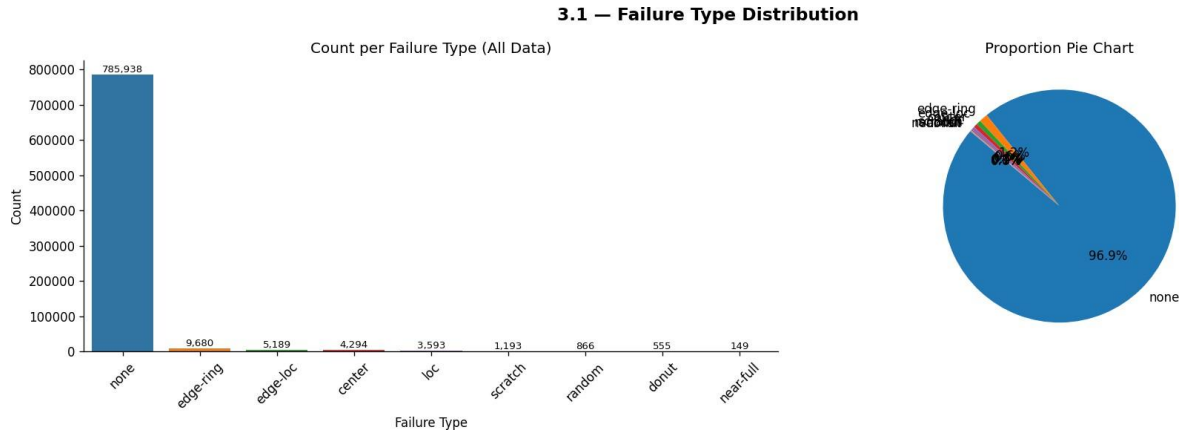


Figure 17: Distribution of all 811,457 maps including the none class (left) and class proportions (right).

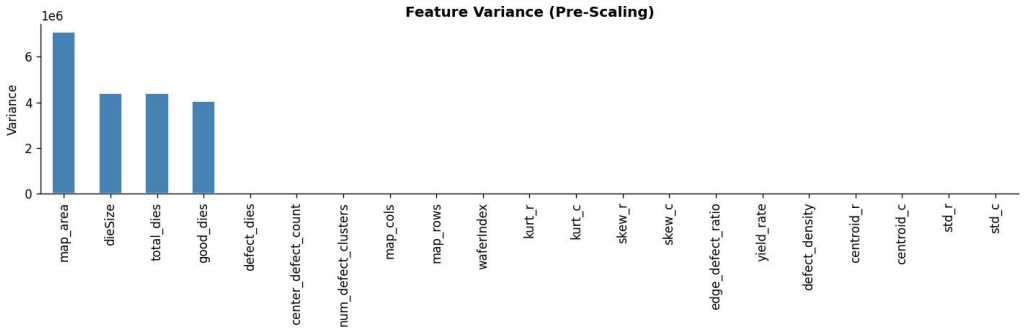


Figure 18: Feature variance before scaling; geometry fields dominate the raw variance, which motivates standardization.

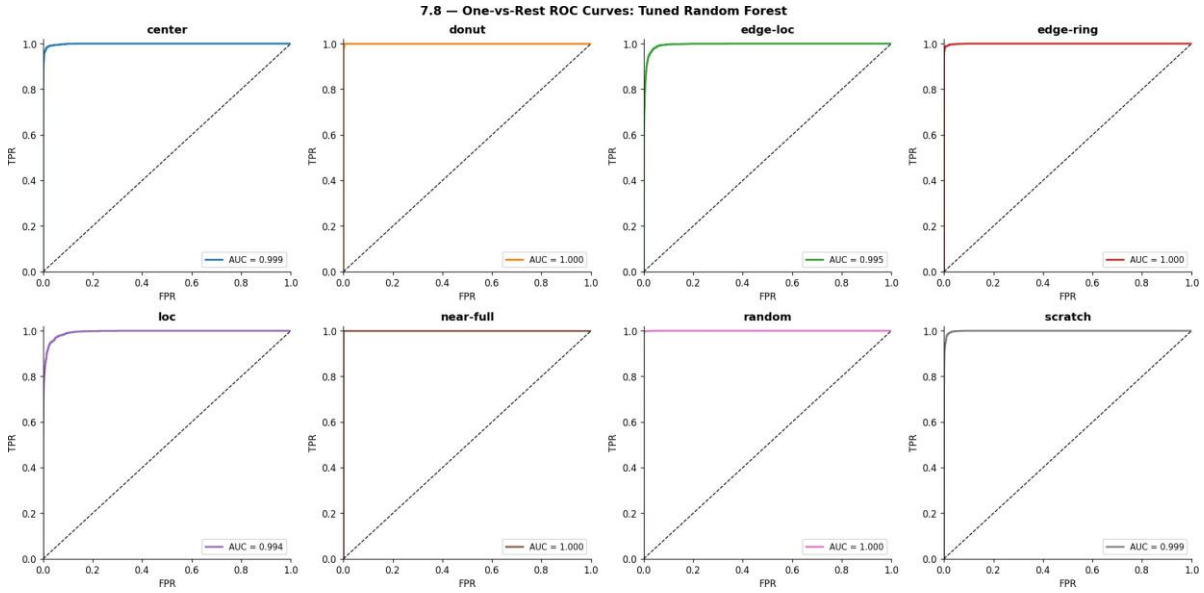


Figure 19: One-vs-rest ROC curves of the tuned Random Forest for each class.

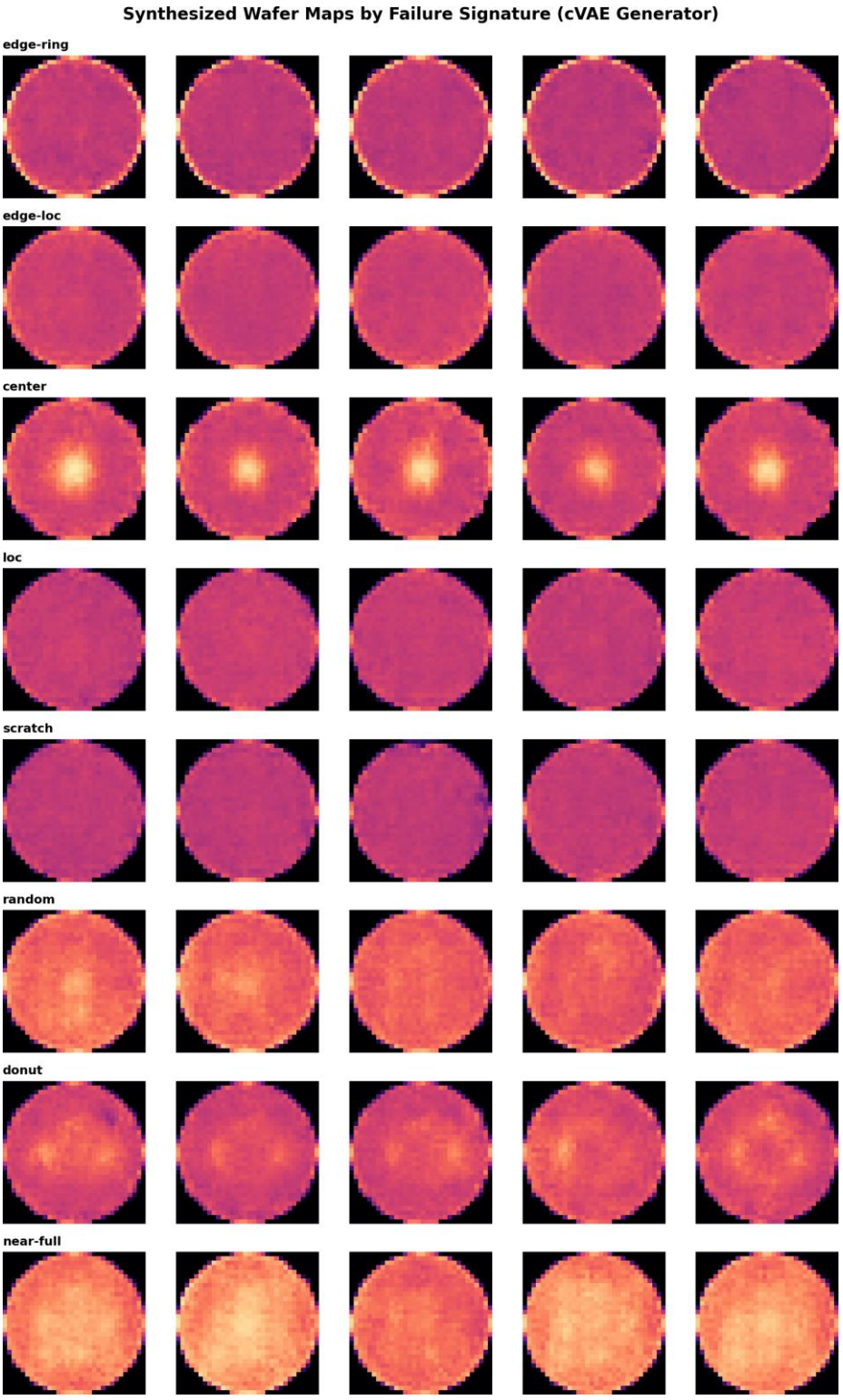


Figure 20: Output of `generate.py --all-classes`. This grid appears less converged than the epoch-25 grid in Figure 14.

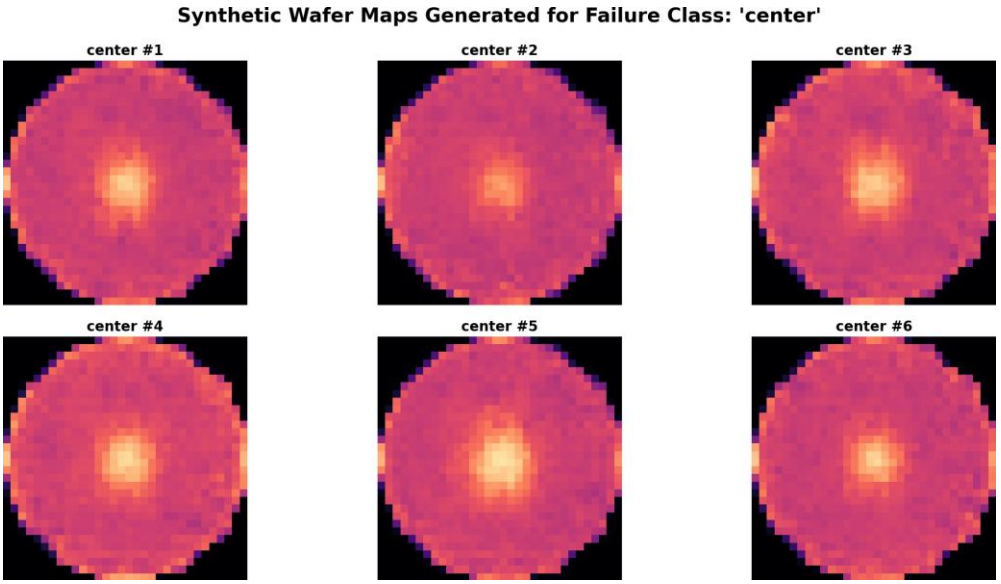


Figure 21: Output of `generate.py -class center -count 6`.

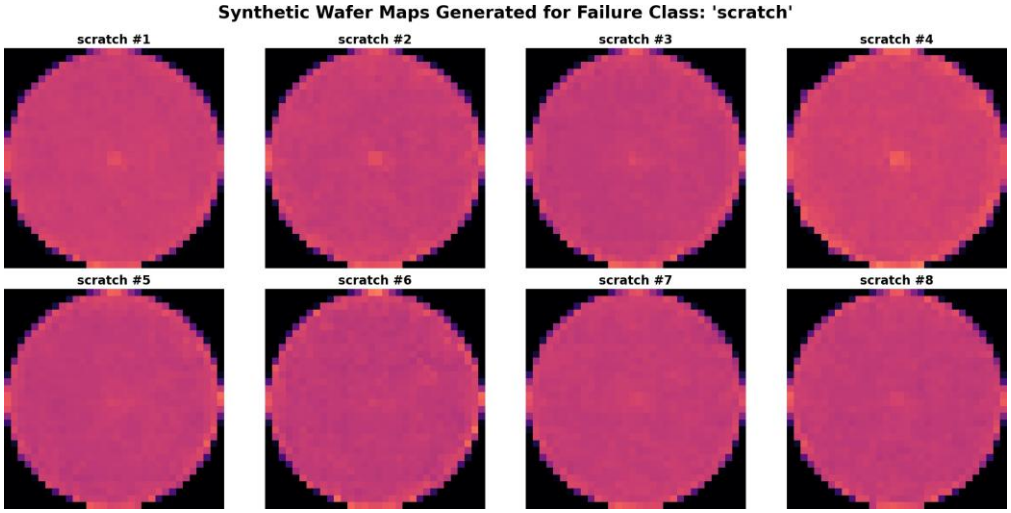


Figure 22: Output of `generate.py -class scratch -count 8`. No linear scratch structure is reproduced, illustrating the limitation discussed in Section 4.

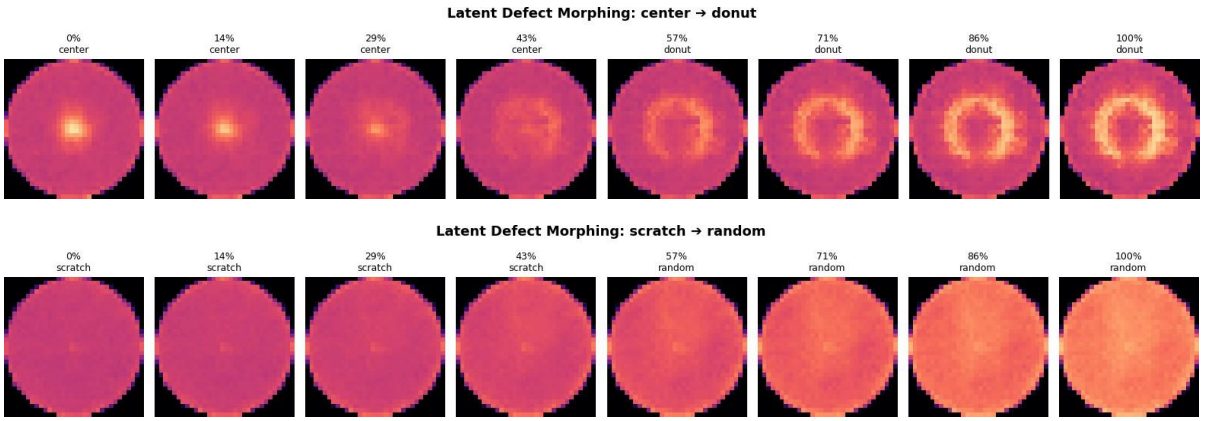


Figure 23: Additional latent morphs from the notebook (top: `edge-ring` to `edge-loc`; bottom: `scratch` to `random`).

B Code Structure and Repository

The implementation is organized as follows.

`vafermap_fixed.ipynb` - Model 1: loading, EDA, feature engineering, SMOTETomek, PCA/t-SNE, classification.

`dl/dataset.py` – loading of `LsWhD.pkl`, label extraction, 32×32 resizing, `.npz` caching, PyTorch `Dataset` and `DataLoader`.

`dl/models/cvae.py` - the conditional VAE (and `cgan.py` for the conditional GAN).

`dl/train_cvae.py` - training with checkpointing and periodic generation grids.

`dl/generate.py` – generation CLI (class, all-classes, morph, export-dataset).

`dl/utils.py` - plotting of wafer grids, class matrices and interpolation strips.

`dl/vafer_generative_dl.ipynb` - interactive notebook for Model 2.

Typical commands:

```
python dl/train_cvae.py --epochs 25 --batch-size 128 --beta 0.5
python dl/generate.py --class center --count 8 --output dl/outputs/center_defect.png
python dl/generate.py --all-classes --count 5 --output dl/outputs/all_classes.png
python dl/generate.py --morph --from-class center --to-class donut --steps 8
python dl/generate.py --export-dataset --samples-per-class 500 --output dl/data/synthetic_aug
```

Dataset: WM-811K, https://vvv.kaggle.com/datasets/qingyi/vm811k-vafer-map.
GitHub repository: https://github.com/1037kavin/Signature-Identification